

TAD PILA: GUÍA COMPLETA EN PYTHON

SÍNTESIS PARA CLASE — CONCEPTOS, REPRESENTACIÓN Y ANÁLISIS COMPLETO

1. Concepto Fundamentales y Política LIFO

Una **Pila (Stack)** es un tipo abstracto de datos que consiste en una secuencia de cero o más elementos que crecen y se contraen siguiendo una política rigurosa de **Último en Entrar, Primero en Salir (LIFO - Last In, First Out)** [1, 23]. Dado que entre los elementos de la Pila existe un orden temporal determinado por el orden de sus inserciones, los datos se recuperan exactamente en el orden inverso al que fueron ingresados [1, 23].

La Analogía de las Bandejas: Imagina una pila de bandejas en un comedor. La última bandeja colocada en la cima es físicamente la primera en ser retirada para su uso [3]. De igual manera, el único extremo accesible para insertar (apilar) o suprimir (desapilar) elementos en una pila de datos se conoce técnicamente como el **tope o cima** [4, 24].

Especificación Formal de Operaciones Abstractas

Un TAD Pila se define matemáticamente mediante las siguientes operaciones teóricas independientes de la implementación [7, 26]:

Operación	Descripción	Entrada/Salida Formal
Crear(P)	Inicializa la pila P como una secuencia vacía [6, 26].	Salida: P = ()
Vacía(P)	Evalúa si la pila P no contiene elementos activos [6, 26].	Entrada: P. Salida: Verdadero si P = (), Falso en caso contrario.
Insertar(P, X)	Ingresa el elemento X en el extremo del tope de P [6, 25].	Entrada: P, X. Salida: P = (a1, a2, ..., an, X)
Suprimir(P, X)	Elimina el elemento del tope de P (el insertado más recientemente) [6, 25].	Entrada: P. Salida: P = (a1, a2, ..., an-1) y X = an
Recorrer(P)	Procesa secuencialmente todos los elementos de P bajo la política LIFO [6, 26].	Entrada: P. Salida: Procesamiento secuencial (ej. mostrar en pantalla).

2. Comparación de Representaciones de Memoria

Existen dos filosofías principales para representar un TAD Pila en memoria: la **secuencial** (estática/contigua) y la **encadenada** (dinámica/dispersa) [13, 14]. La elección óptima depende de la variabilidad y escala del volumen de datos [15].

Característica	Representación Secuencial	Representación Encadenada
Estructura de Datos	Registro compuesto por un arreglo contiguo de tamaño estático y un índice entero tope [14].	Estructura de celdas dispersas con un enlace/puntero sig y un puntero tope al nodo superior [14, 15].
Capacidad Física	Acotada. Requiere definir un tamaño máximo al inicializar. Puede producir desbordamiento (<i>Stack Overflow</i>) [17].	Ilimitada en la teoría. Crece dinámicamente según la memoria disponible del sistema [15, 17].

Característica	Representación Secuencial	Representación Encadenada
Uso de Memoria	Eficiente en espacio neto por elemento, pero puede haber desperdicio si el tamaño reservado es muy superior al usado.	Mayor consumo por elemento debido al almacenamiento del puntero o dirección de enlace (<i>overhead</i>) [15].
Eficiencia Operativa	Acceso directo extremadamente veloz. Sin sobrecarga de asignación en tiempo de ejecución.	Ligera sobrecarga temporal por operaciones frecuentes de creación (<i>new</i>) y eliminación de celdas.

3. Implementación Secuencial en Python

En lenguajes de bajo nivel como C++, la pila secuencial requiere de un arreglo y un entero indexador tope [14]. A continuación, presentamos cómo imitar de manera **estricta** este comportamiento físico utilizando una lista de tamaño fijo en Python para preservar el rigor del diseño conceptual de la cátedra [17]. Seguidamente, se expone la versión **idiomática de Python** que simplifica este proceso utilizando métodos nativos de listas dinámicas [30].

Código Python: Representación Secuencial Estricta (Simulación de Arreglo)

```
class PilaSecuencial:
    def __init__(self, cant=0):
        self.cant = cant          # Capacidad estática física
        self.tope = -1           # -1 indica que la pila está vacía
        self.items = [None] * cant # Simulación del arreglo contiguo

    def vacia(self):
        return self.tope == -1

    def insertar(self, x):
        if self.tope < self.cant - 1:
            self.tope += 1
            self.items[self.tope] = x
            return x
        else:
            return 0 # Desbordamiento de capacidad (Cátedra)

    def suprimir(self):
        if self.vacia():
            print("Pila vacia")
            return 0
        else:
            x = self.items[self.tope]
            self.items[self.tope] = None # Limpieza opcional de referencia
            self.tope -= 1
            return x

    def mostrar(self):
        if not self.vacia():
            for i in range(self.tope, -1, -1):
                print(self.items[i])
```

Código Python: Representación Idiomática (Lista Dinámica)

En la práctica real con Python, la implementación de una pila se simplifica drásticamente gracias a que las listas nativas son dinámicas y ya proveen operaciones con rendimiento amortizado constante para inserciones y eliminaciones al final [30]:

```
class Pila:
    def __init__(self):
        self.items = []          # Lista dinámica nativa de Python

    def isVacia(self):
        return self.items == []

    def apilar(self, item):
        self.items.append(item)  # Agrega al tope de la pila

    def desapilar(self):
        if self.isVacia():
            raise IndexError("Desapilar de una pila vacia")
        return self.items.pop()  # Remueve y retorna del tope

    def longitud(self):
        return len(self.items)
```

Nota de diseño: La implementación idiomática en Python utiliza `append()` y `pop()`, los cuales operan internamente en el extremo derecho de la lista. Esto coincide directamente con la convención abstracta de ubicar el tope al extremo derecho de la secuencia para minimizar el movimiento físico de datos en memoria [4, 30].

4. Implementación Encadenada en Python

La representación encadenada utiliza objetos de tipo **Celda** que se enlazan dinámicamente mediante referencias de objetos en Python [19, 20]. A diferencia de C++, Python cuenta con un **Recolector de Basura automático** (*Garbage Collector*) basado en conteo de referencias. Esto significa que cuando se desapila un elemento y se remueve su referencia del puntero tope, la memoria de la celda desapilada es liberada automáticamente por el intérprete sin necesidad de llamar de manera explícita a funciones como `free()` o `delete` [21, 32].

Código Python: Representación Encadenada Dinámica

```

class Celda:
    def __init__(self, item=None, sig=None):
        self.item = item
        self.sig = sig

    def obteneritem(self):
        return self.item

    def cargaritem(self, xitem):
        self.item = xitem

    def cargarsig(self, xtope):
        self.sig = xtope

    def obtenersig(self):
        return self.sig

class PilaEncadenada:
    def __init__(self):
        self.cant = 0
        self.tope = None # Referencia al nodo de la cima (Celda)

    def vacia(self):
        return self.cant == 0

    def insertar(self, x):
        ps1 = Celda()
        ps1.cargaritem(x)
        ps1.cargarsig(self.tope)
        self.tope = ps1
        self.cant += 1
        return ps1.obteneritem()

    def suprimir(self):
        if self.vacia():
            print("Pila vacia")
            return 0
        else:
            aux = self.tope
            x = self.tope.obteneritem()
            self.tope = self.tope.obtenersig()
            self.cant -= 1
            # Python destruirá el objeto 'aux' automáticamente al perder sus referencias
            del aux # Eliminación explícita para emular la cátedra
            return x

    def muestratope(self):
        if not self.vacia():
            return self.tope.obteneritem()
        return None

    def recorrer(self, aux):
        if aux is not None:
            print(aux.obteneritem())
            self.recorrer(aux.obtenersig())

```

Clave de Cátedra: ¿Por qué enlazamos los nodos desde el tope hacia la base ($a_n \rightarrow a_{n-1} \rightarrow \dots \rightarrow a_1$) y no de forma inversa? Porque si el tope apuntara a la base, al suprimir un elemento no tendríamos una forma directa de encontrar la celda inmediatamente anterior sin tener que recorrer toda la estructura de forma lineal ($O(n)$), perdiendo la eficiencia constante $O(1)$ que caracteriza a la pila [15, 16].

5. Aplicación: Correspondencia de Delimitadores

Un uso fundamental del TAD Pila es el control de correspondencia de símbolos de agrupación (paréntesis (), llaves { } y corchetes []) en expresiones aritméticas [8]. La naturaleza LIFO de la pila permite asegurar que el último símbolo de apertura ingresado sea obligatoriamente el primero en cerrarse [11, 24].

Algoritmo de Verificación en Python

```
def verificar_correspondencia(expresion):
    pila = Pila() # Utiliza la versión de pila idiomática
    error = False

    for caracter in expresion:
        # Si es un delimitador de apertura, se apila
        if caracter in '[' , '{' , '(' :
            pila.apilar(caracter)

        # Si es un delimitador de cierre, se evalúa
        elif caracter in ']' , '}' , ')' :
            if pila.isVacia():
                error = True
                break

            aux = pila.desapilar()
            # Validación de correspondencia exacta
            if ((caracter == ']' and aux != '[') or
                (caracter == '}' and aux != '{') or
                (caracter == ')' and aux != '(')):
                error = True
                break

    if not pila.isVacia() or error:
        return "ERROR DE CORRESPONDENCIA"
    else:
        return "CORRESPONDENCIA"

# Pruebas con ejemplos de la cátedra
print(verificar_correspondencia("{[(A-B)*C]^D}")) # CORRESPONDENCIA
print(verificar_correspondencia("{([A-B)*C]^D}")) # ERROR DE CORRESPONDENCIA
```

6. Análisis de Complejidad y Eficiencia

En ambas representaciones, las operaciones de modificación de la pila operan de manera local sobre un extremo fijo (el tope) [4]. Esto garantiza un rendimiento algorítmico sumamente eficiente.

Eficiencia Teórica de Operaciones en Pilas

Operación	Representación Secuencial	Representación Encadenada
Crear / Inicializar	Theta(1) - Costo Constante	Theta(1) - Costo Constante
Insertar (Push)	Theta(1) - Costo Constante	Theta(1) - Costo Constante
Suprimir (Pop)	Theta(1) - Costo Constante	Theta(1) - Costo Constante
Vacía (Check)	Theta(1) - Costo Constante	Theta(1) - Costo Constante

Operación	Representación Secuencial	Representación Encadenada
Recorrer / Mostrar	Theta(n) - Costo Lineal	Theta(n) - Costo Lineal

Concepto Clave: Análisis Amortizado

El **análisis amortizado** evalúa el tiempo promedio requerido para ejecutar una secuencia completa de N operaciones sobre una estructura de datos [32]. A diferencia de un análisis tradicional en el peor caso —que asume de forma pesimista que cada operación individual toma el tiempo máximo posible—, el análisis amortizado reconoce que las operaciones costosas ocurren solo de forma ocasional y que su costo se compensa (amortiza) por el bajo costo de las operaciones restantes de la secuencia [32, 51]. Por ejemplo, en una lista dinámica de Python, duplicar el tamaño del arreglo interno cuando este se llena es una operación lineal $O(n)$, pero ocurre tan raramente que la operación de inserción sigue teniendo un costo amortizado constante de **Theta(1)** en promedio.